

approach is now making its way into embedded system design.

The advantage is that components that are slow or resource-intensive can be optimised, moved to hardware, or adjusted. Traditionally, breaking a system into hardware and software has been a complex and time-consuming process.

Generative AI automates many of these steps. Starting from a textual description, large language models infer an internal representation that the machine can understand, enabling faster translation from requirements to functional system design.

That intermediate representation can then be simulated on available microcontrollers to identify bottlenecks or power challenges. Based on this feedback, the system can be fine-tuned iteratively, with iterations largely handled by the machine. Existing solutions already manage about 85 to 90 per cent of this process, which is sufficient to save significant engineering effort and reduce time to market.

From text to silicon

Once the system is roughly divided into hardware and software, the next step is to build and validate these components. The hardware is developed, the software is executed on it, and adjustments are made as necessary, refining the system as the design moves to the next stage.

The next level involves hardware schematics, where the functional description generated earlier serves as the foundation for the design. Based on this description, generative AI can help create initial schematic designs using learned libraries and prior design knowledge.

From these schematics, combined with the corresponding software, a complete system can be created. AI can also support the transition from schematic to PCB layout. Many tools already assist in this step, and some are freely

available as developers continue to improve their solutions. To some extent, these tools also support schematic entry and can handle first- or second-level PCB layout. While they are not yet mature enough to produce a fully commercial-grade PCB, they still save significant effort and time.

In parallel, these tools optimise performance during hardware-software partitioning and while refining the schematic. Thermal management is another key aspect of this stage. Projects can fail due to inadequate thermal design. For example, a projector engine developed over 20 years ago generated significant heat, causing the commercial product to overheat—a challenge many engineers may have encountered. Generative AI can assist by assessing heat generation from previous designs and predicting thermal challenges as designs move from schematic to PCB, making thermal management a valuable byproduct of the workflow.

When AI brews your coffee

To illustrate this, a simple vending machine example was evaluated. The first step was a textual description of the target functionality, with sufficient detail provided for the machine to interpret the requirements.

The generative AI engine then created a flowchart, converting the internal representation into a human-readable graphical form. Using this flowchart, along with its interpretation of the textual description, the engine generated executable C code with comments explaining the functionality of each part.

While documentation is often generated from code using tools such as Doxygen, this approach enables starting from simple language and producing near-commercial C code, along with readable documentation. This workflow highlights a direction worth further exploration.

Using a coffee vending machine as an example, the system begins in an idle state with a menu that includes options such as espresso and cappuccino. Once a drink is selected, the machine requests further customisation. The process includes choices such as additional sugar, milk, or coffee, and then proceeds to payment. If the payment is successful, the system prepares the bill; if not, it automatically prompts again, despite this behaviour not being explicitly defined in the requirements.

The system also handles multiple orders. When several users are placing orders, it loops back to the menu or returns to idle depending on user choice. All of these behaviours are generated by the AI without detailed instructions.

The generated output includes all states represented as a DOT edge graph, along with a corresponding C file that uses a switch-case structure to implement the state of machine. Functions are modularised and commented, with examples such as `handleBrowseMenu` clearly illustrating actions and selection handling. Interestingly, the AI also infers item costs. It assigns espresso a lower cost than cappuccino and latte, and black coffee a lower cost than espresso, without any prices being specified.

The generated documentation provides a complete view of the system, covering all function calls and flows. For debugging or verification, approximately 50 to 60 per cent of the work is already completed automatically. Several large language models and engines were tested, including Gemini 2.5 and Meta's LLaMA models. Most produced similar results, with Gemini 2.5 performing best in these evaluations.

From a business perspective, AI reduces engineering and R&D effort, minimises errors arising from repetitive coding, and accelerates time-to-market. Engineers can make